SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

---

1. Introduction

1.1. Purpose

The purpose of this System Analysis Document (SAD) is to comprehensively highlight the technical scope, architectural boundaries, and design-related decisions behind the development of the **AutoYield** kernel. This document transitions the strategic goals outlined in the Product Requirement Document (PRD) into actionable engineering blueprints.

The key elements of the system covered in this document are:

1. **Architecture:** AutoYield is a cloud-native, event-driven platform. The architecture separates the user-facing interface (React) from the stateful transaction backend (FastAPI/Supabase) and the cognitive reasoning layer (LangGraph + Z.AI GLM). These modules operate asynchronously to prevent computational bottlenecks during heavy LLM inference.

2. **Data Flows:** Illustrates how unstructured inputs (blurry invoices) or internal database anomalies flow through the ingestion layer, get processed by the Dual-Agent decision engine, and result in persistent Database Operations (CRUD) that immediately update the live frontend via Server-Sent Events (SSE) and Supabase Realtime websockets.

3. **Model Process:** Details the end-to-end workflow of the core autonomous features: detecting inventory anomalies, triggering the P-Agent vs. R-Agent debate, resolving the strategy, and autonomously modifying live menus or executing procurement orders.

4. **Role of Reference:** This document serves as the absolute source of truth for the development and QA teams, outlining API contracts, database schemas (3NF), LLM constraint boundaries, and the orchestration of four specialized LangGraph sub-graphs

(Conversation + Debate, Demand Forecasting, Proactive Crisis, Invoice Ingestion) for the 10-day hackathon sprint.

1.2. Background

Traditional Food and Beverage (F&B) management is inherently reactive and crippled by "unstructured data blindspots."

1. **Previous Version (The Status Quo):** Existing restaurant software (StoreHub, Food Market Hub) acts strictly as static digital ledgers. They accurately record what happened but require human managers to act as the bridge between external shocks (e.g., a supplier texting about a 30% price hike) and internal adaptation (e.g., rewriting the menu).

2. **Changes in Major Architectural Components:** AutoYield replaces the human "data-entry bridge" with a proactive AI agent. By shifting from a monolithic CRUD application to a Multi-Agent System (MAS), the architecture now includes a "Cognitive Orchestrator" that sits above the database, monitoring state changes and executing yield-optimizing actions autonomously.

3. **New Capabilities:** AutoYield introduces autonomous unstructured asset ingestion (OCR to structured JSON), a God-Mode testing sandbox with live parameter manipulation, a Dual-Agent economic loop that debates Profit vs. Customer Risk in real-time, a time-aware simulation engine (independent simulated clock), and a proactive background monitor that polls inventory/orders every 5 seconds to trigger autonomous crisis responses.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations

Restaurant Manager / Owner	Oversees restaurant profitability, approves/rejects high-stakes agent decisions, and monitors macro trends.	Expects a reduction in administrative cognitive load, immediate anomaly detection, and a transparent "thought trace" explaining why the AI made a decision.
Kitchen Staff (BoH)	Prepares food and manages physical inventory.	Expects the digital menu to perfectly sync with their physical fridge reality to avoid fulfilling orders for out-of-stock items.
Customers (FoH)	Interacts with the dynamic digital menu to place orders.	Expects an updated, accurate menu with optimized pricing and intelligent promotional bundles.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Web-based Management Dashboard (Admin) & Dynamic Menu UI (Customer)
Architecture	Client-Server API model with an asynchronous AI Microservice (four LangGraph sub-graphs) & Realtime Websockets.

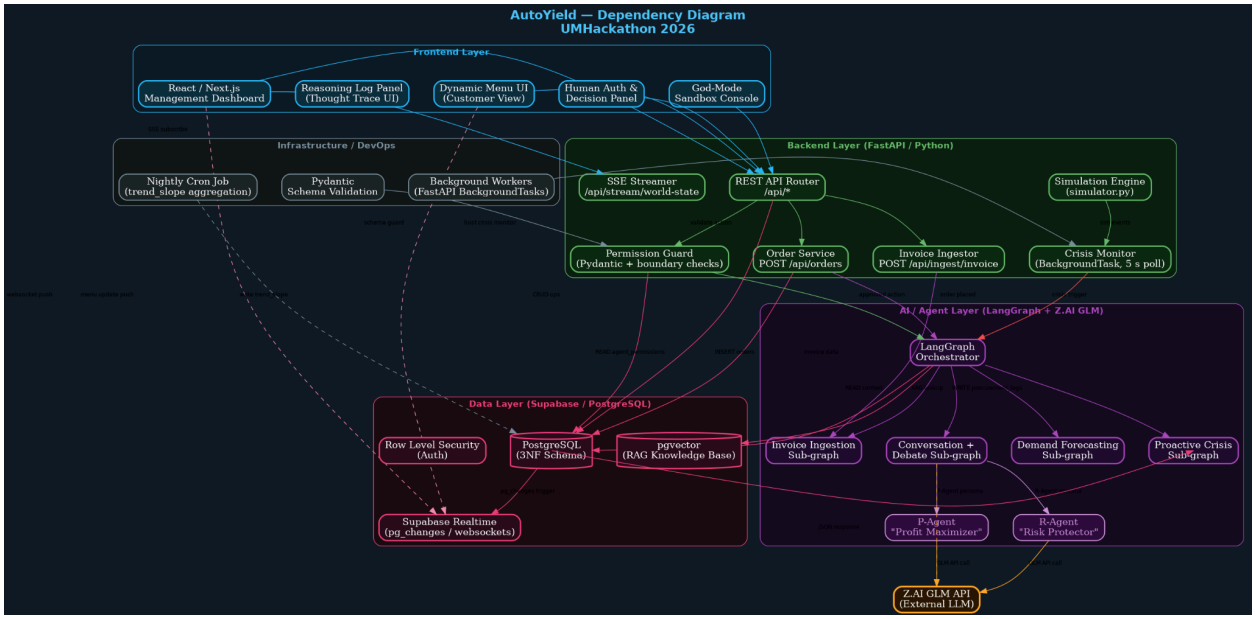
AutoYield operates as a decoupled, modern web application. The frontend (React) communicates with the backend (FastAPI) via standard RESTful APIs for transactional data. Crucially, live updates (menu changes, order queues) are pushed to the client using Supabase Realtime and Server-Sent Events (SSE). The "Brain" of the system consists of four LangGraph sub-graphs. The Proactive Crisis sub-graph is triggered by a background monitor that polls the database every 5 seconds for anomalies (low stock, order surge). Other sub-graphs are invoked via FastAPI endpoints or scheduled tasks.

2.1.2. LLM as Service Layer

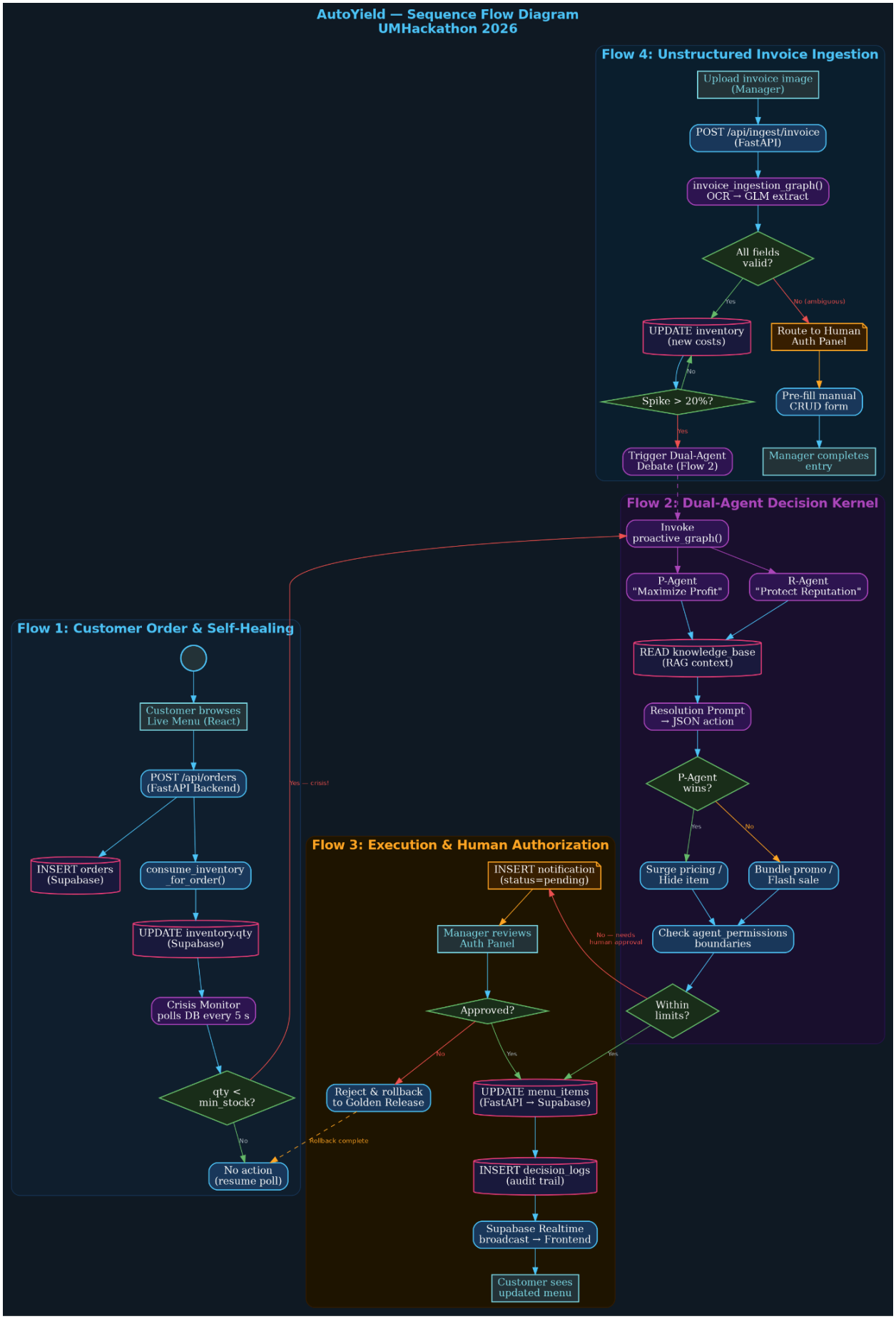
The Z.AI GLM is not just an API call; it acts as the **Service Layer Orchestrator**.

Instead of hard-coded if/else statements for pricing logic, the backend wraps the GLM in a LangGraph framework. When an anomaly is detected, the appropriate sub-graph is invoked. The LLM receives persona prompts (P-Agent and R-Agent) and is bound to a curated set of atomic tools (e.g., simulate_yield_scenario, check_operational_capacity, execute_operational_action). The LLM returns tool calls, which the LangGraph executor runs, and the results are fed back into the graph.

2.1.3. Dependency Diagram & Sequence Flow

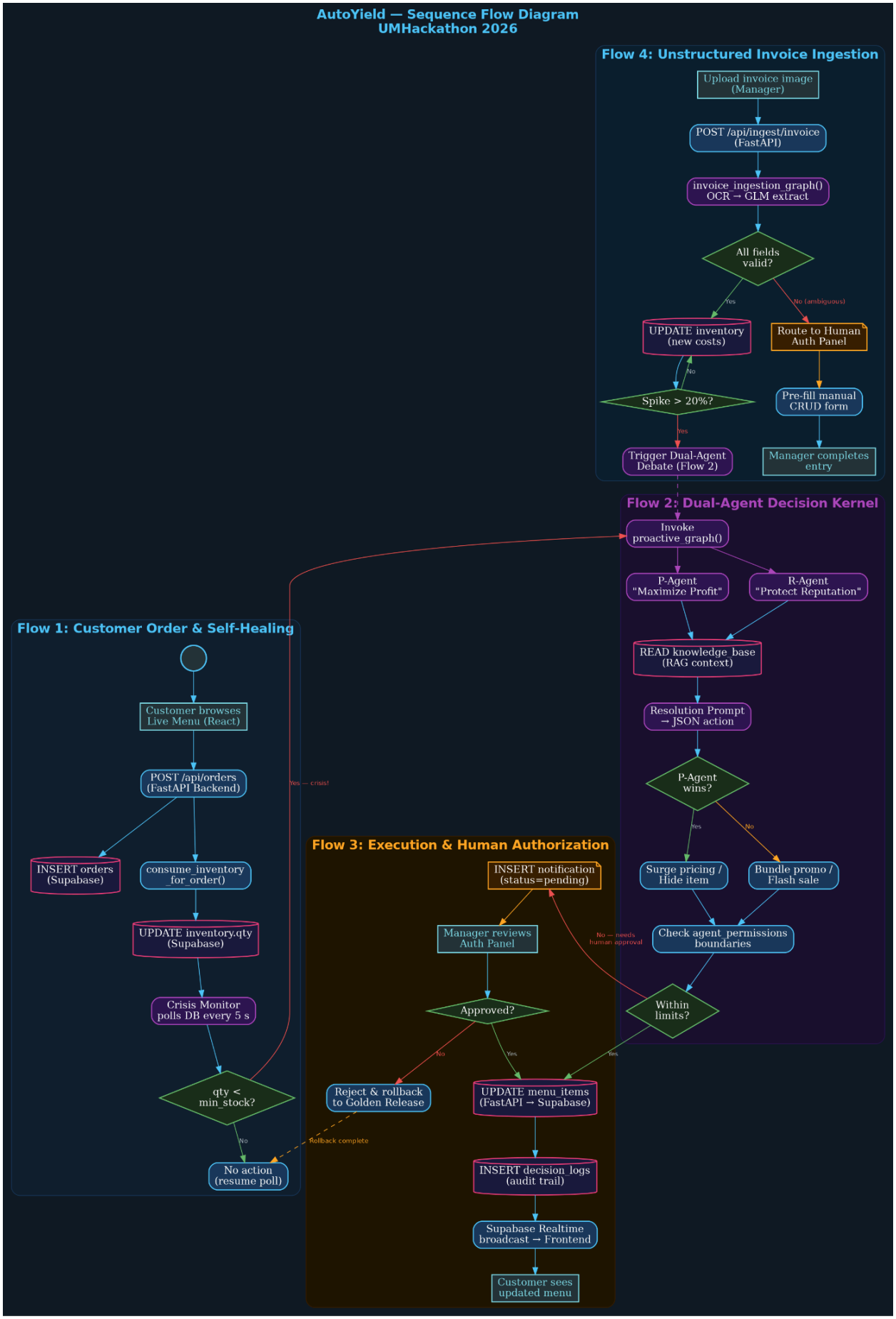


Dependency Diagram


AutoYield — Sequence Flow Diagram
UMHackathon 2026
Flow 4: Unstructured Invoice Ingestion
Upload invoice image (Manager)
POST /api/ingest/invoice (FastAPI)
invoice_ingestion_graph() OCR → GLM extract
All fields valid?
Yes
No (ambiguous)
UPDATE inventory (new costs)
Route to Human Auth Panel
No
Spike > 20%?
Pre-fill manual CRUD form
Yes
Trigger Dual-Agent Debate (Flow 2)
Manager completes entry
Flow 2: Dual-Agent Decision Kernel
Invoke proactive_graph()
P-Agent "Maximize Profit"
R-Agent "Protect Reputation"
READ knowledge_base (RAG context)
Resolution Prompt → JSON action
P-Agent wins?
Yes
No
Surge pricing / Hide item
Bundle promo / Flash sale
Check agent_permissions boundaries
Within limits?
Yes
No — needs human approval
Flow 1: Customer Order & Self-Healing
Customer browses Live Menu (React)
POST /api/orders (FastAPI Backend)
INSERT orders (Supabase)
consume_inventory_for_order()
UPDATE inventory.qty (Supabase)
Crisis Monitor polls DB every 5 s
qty < min_stock?
Yes — crisis!
No
No action (resume poll)
Flow 3: Execution & Human Authorization
INSERT notification (status=pending)
Manager reviews Auth Panel
Approved?
No
Yes
Reject & rollback to Golden Release
Rollback complete
UPDATE menu_items (FastAPI → Supabase)
INSERT decision_logs (audit trail)
Supabase Realtime broadcast → Frontend
Customer sees updated menu

sequence flow diagram

2.1.4. Sequence Diagram (Place Order & Self-Healing Flow)


AutoYield — Sequence Flow Diagram
UMHackathon 2026
Flow 4: Unstructured Invoice Ingestion
Upload invoice image (Manager)
POST /api/ingest/invoice (FastAPI)
invoice_ingestion_graph() OCR → GLM extract
All fields valid?
Yes
No (ambiguous)
UPDATE inventory (new costs)
Route to Human Auth Panel
No
Spike > 20%?
Pre-fill manual CRUD form
Yes
Trigger Dual-Agent Debate (Flow 2)
Manager completes entry
Flow 2: Dual-Agent Decision Kernel
Invoke proactive_graph()
P-Agent "Maximize Profit"
R-Agent "Protect Reputation"
READ knowledge_base (RAG context)
Resolution Prompt → JSON action
P-Agent wins?
Yes
No
Surge pricing / Hide item
Bundle promo / Flash sale
Check agent_permissions boundaries
Within limits?
Yes
No — needs human approval
Flow 1: Customer Order & Self-Healing
Customer browses Live Menu (React)
POST /api/orders (FastAPI Backend)
INSERT orders (Supabase)
consume_inventory_for_order()
UPDATE inventory.qty (Supabase)
Crisis Monitor polls DB every 5 s
qty < min_stock?
Yes — crisis!
No
No action (resume poll)
Flow 3: Execution & Human Authorization
INSERT notification (status=pending)
Manager reviews Auth Panel
Approved?
No
Yes
Reject & rollback to Golden Release
Rollback complete
UPDATE menu_items (FastAPI → Supabase)
INSERT decision_logs (audit trail)
Supabase Realtime broadcast → Frontend
Customer sees updated menu

Sequence Diagram

2.2. Technological Stack

- **2.2.1. Frontend: React.js** . Chosen for robust state management and seamless integration with Supabase Realtime subscriptions to reflect agentic actions instantly without page reloads.

- **2.2.2. Backend: FastAPI (Python)**. Chosen due to its native asynchronous support, auto-generated OpenAPI documentation, and seamless integration with Python-based AI libraries (LangChain/LangGraph) and Pydantic data validation.

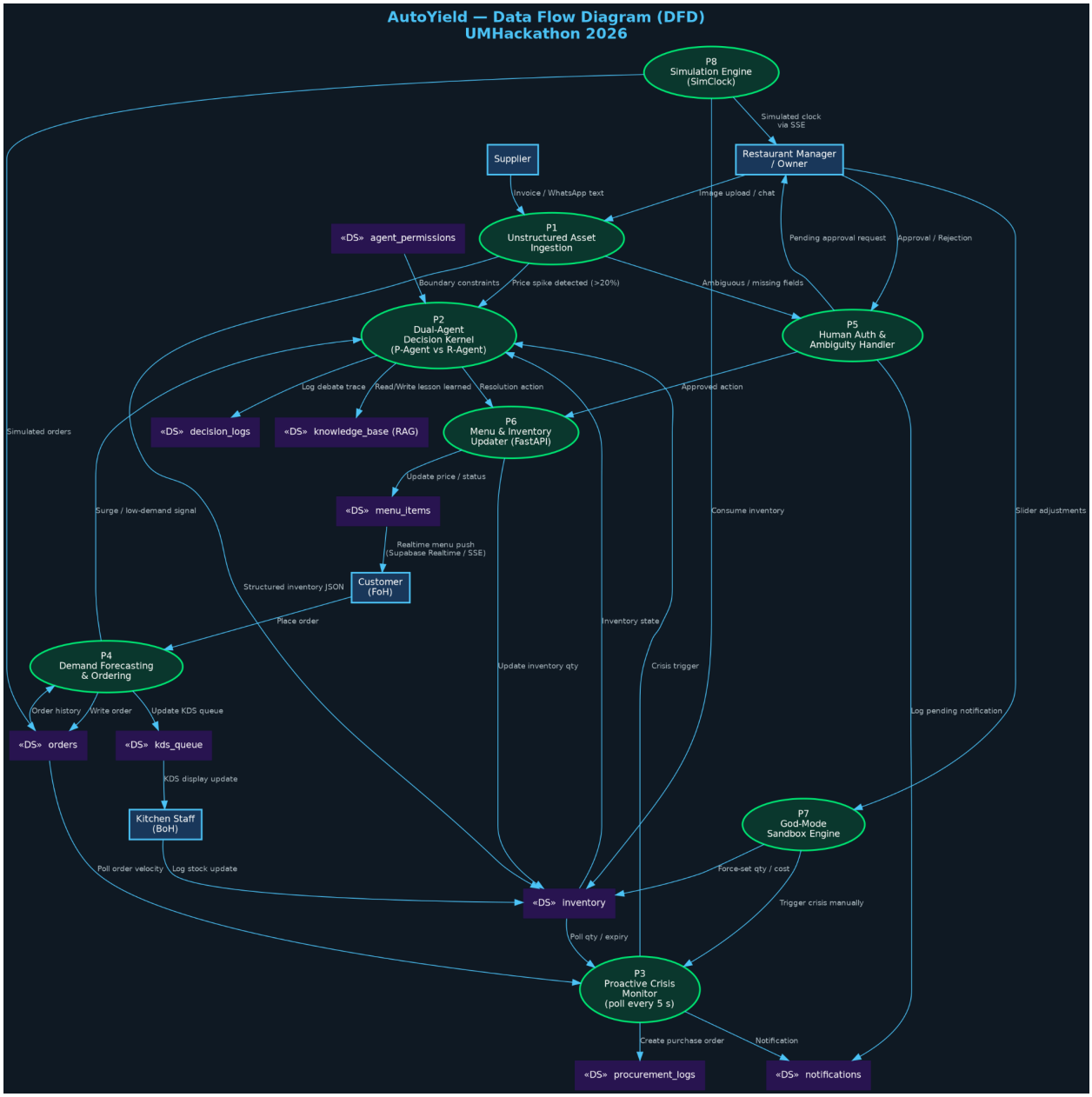
- **2.2.3. AI / Agent Layer: LangGraph + Z.AI GLM API + Google gemini-2.5-flash (for image handling)**. LangGraph manages the stateful, cyclic graphs required for the Dual-Agent debate, while Z.AI GLM provides the raw reasoning power.

- **2.2.4. Database & Realtime: Supabase (PostgreSQL)**. Chosen for its out-of-the-box realtime websocket broadcasting (pg_changes), Row Level Security, and pgvector support for the RAG Knowledge Base.

2.3. Key Data Flows

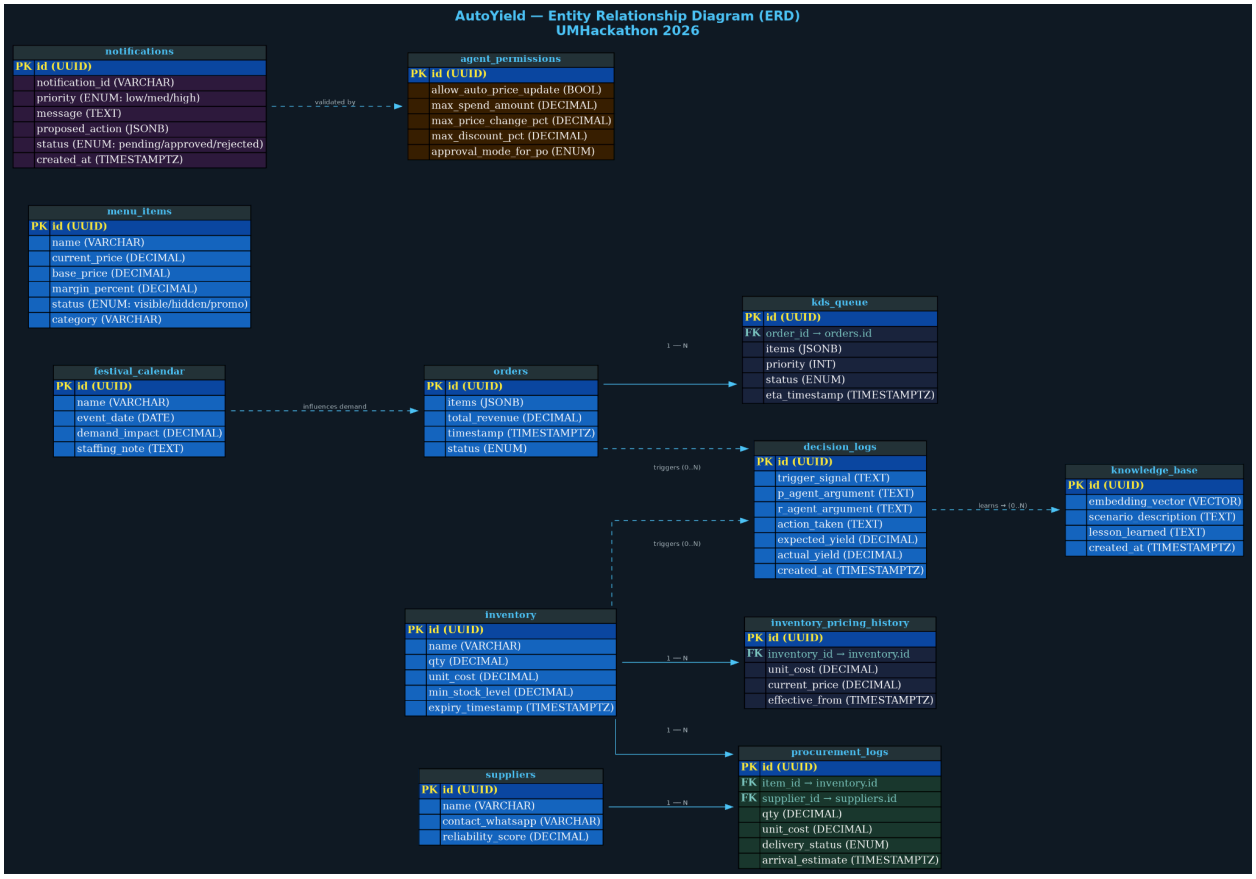
2.3.1. Data Flow Diagram (DFD) Analysis

Based on the system architecture diagram provided, AutoYield categorizes data flows into 5 distinct operational pathways:



Data Flow Diagram

2.3.2. Normalized Database Schema (Entity Relationship)



3. Functional Requirements & Scope

3.1. Minimum Viable Product (MVP)

To showcase technical feasibility within the hackathon timeframe, the MVP will strictly focus on the following core features:

	Features	Description
1	**God-Mode Sandbox & Live UI**	A testing dashboard with sliders (Inventory levels, Order velocity) that immediately trigger AI reactions visible on a real-time order queue and live digital menu.

2	**Dual-Agent Decision Kernel**	The core reasoning engine where P-Agent (Profit) and R-Agent (Risk) debate responses to crises (e.g., supply shortages) and autonomously execute menu/price updates.
3	**Unstructured Asset Ingestion**	A chatbot/upload interface that parses messy supplier texts or blurry invoices into structured JSON using the GLM, directly updating the inventory DB.
4	**Human Decision & Auth Panel**	A managerial safety net where high-risk agent suggestions (or ambiguous OCR scans) are held in a "Pending" state until human approval is granted.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	The backend must handle high-frequency order streams simulated by the God-Mode velocity slider without crashing the database.	Use Supabase connection pooling and process AI reasoning asynchronously via background workers (FastAPI BackgroundTasks).

Reliability (Frame Stability)	Autonomous AI decisions must never produce financially ruinous outcomes (e.g., pricing an item at RM0.01).	Backend implements strict mathematical floor/ceiling bounds. Any AI JSON payload violating these bounds is rejected and routed to Human Auth.
Token Latency	GLM reasoning for complex dual-agent debates can be slow. The UI must remain perfectly responsive.	Implement non-blocking Async API calls. The frontend uses SSE and displays a robust "Agent is Thinking..." animation sequence.
Cost Efficiency	Constant LLM polling is financially unviable.	Debounce triggers. The agent only wakes up for "Macro Anomalies" (e.g., >10% price spike). Routine tasks bypass the LLM.

3.3. Out of Scope / Future Enhancements

- **Physical POS Hardware Integration:** We will mock the receipt printing and cash drawer interactions; no hardware APIs will be built.
- **Live Payment Gateways:** Stripe/PayPal integrations are out of scope; all transactions in the simulation are assumed successful.
- **Automated External Messaging Execution:** The system will draft resupply WhatsApp messages but will not autonomously send them to real-world phone numbers to avoid spamming.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

- **4.1.1. Grayscale Rollout & A/B Testing:** During future production deployment, AI autonomous actions will be rolled out in "Read-Only/Suggestion" mode first. Only after the manager accepts 90% of suggestions will the system be allowed to execute autonomously (Full Rollout).

- **4.1.2. Emergency Rollback (ER) & Golden Release:** Every time the AI mutates the menu_items table (e.g., changing prices), the previous state is logged. A global "Kill Switch" button on the manager dashboard instantly reverts the database to the Golden Release (the morning's baseline menu) if the AI hallucinates.

- **4.1.3. Priority Matrix:**
    - **P1 Critical:** AI attempts to set a negative price margin. Action: Immediate rollback, lock agent, alert manager.
    - **P2 High:** Unstructured ingestion fails to find a total price on an invoice. Action: Route to Ambiguity Flow (Human Auth panel).
    - **P3 Medium:** SSE latency exceeds 3 seconds. Action: Frontend degrades gracefully to standard REST polling.

4.2. Monitoring

System health and AI sanity are monitored via the decision_logs table. This serves as an immutable audit trail. The frontend provides a "Reasoning Log Panel" that visualizes this table in real-time, allowing judges to literally "read the mind" of the AI as it evaluates parameters.

4.3. Assumptions

- Users (Managers) possess modern web browsers with enabled Websocket support for real-time SSE syncing.

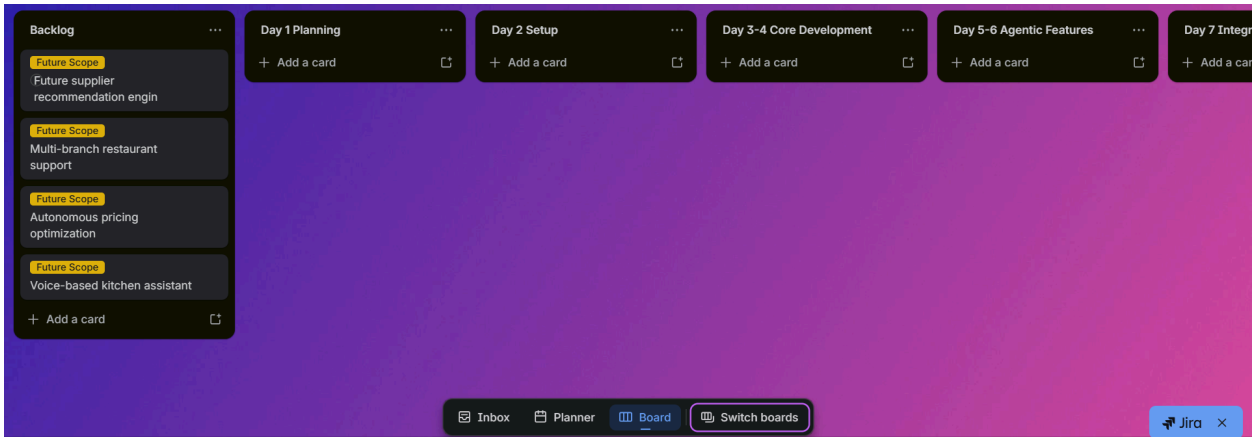
- Hackathon testing environments will have stable internet access to reach the external Z.AI GLM API and Supabase cloud.
- Kitchen staff will accurately log "order complete" statuses in the real world to keep the digital twin's inventory accurate.

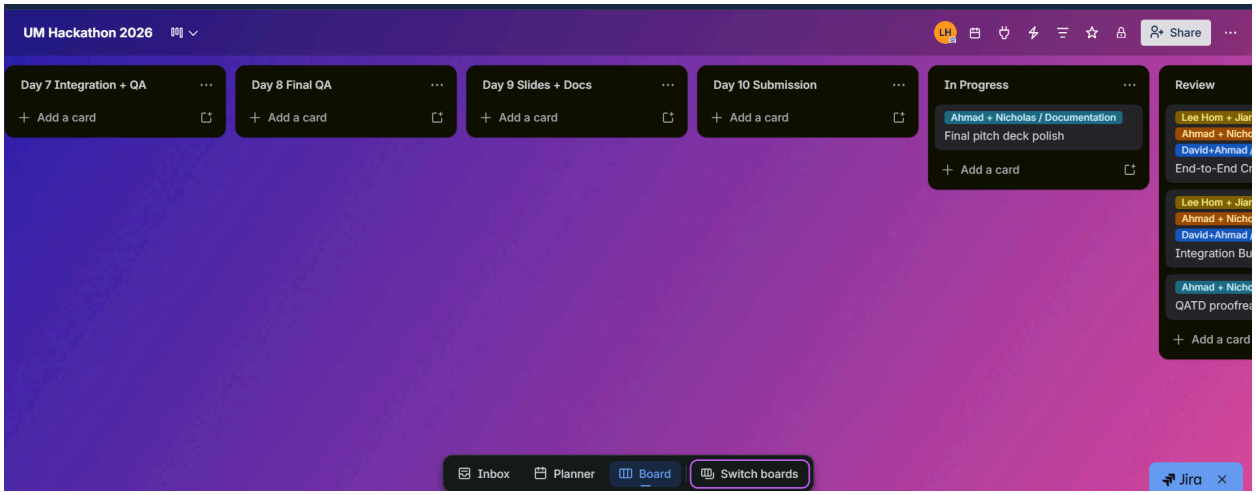
4.4. External Dependencies

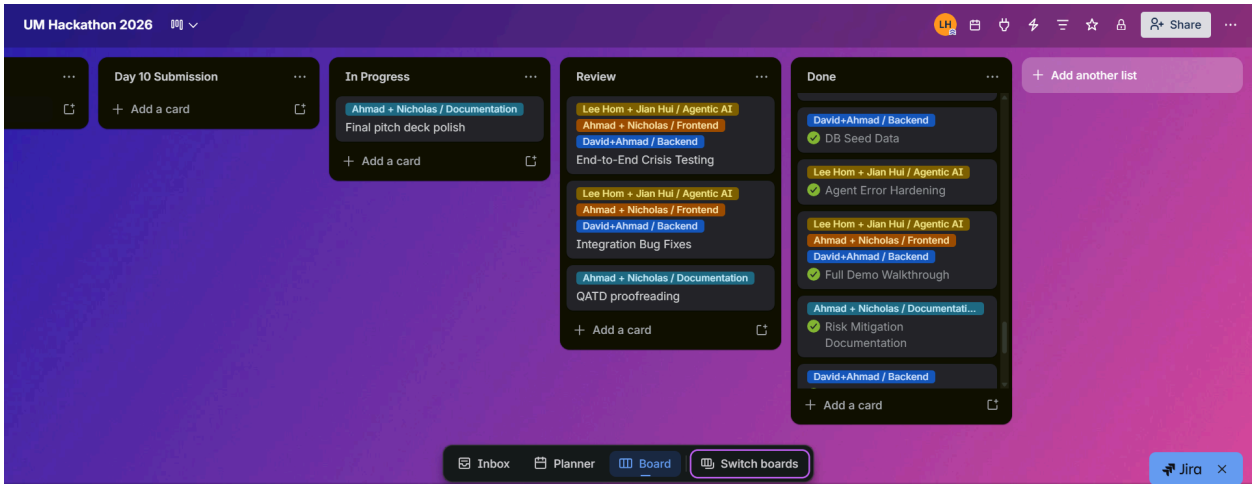
Tools	Purpose	Risks & Mitigation
Z.AI GLM API	Core reasoning engine for unstructured parsing and Dual-Agent debate.	**High Risk:** Rate limits or API downtime. Mitigation: Graceful degradation—if the API fails, the system reverts to a standard, non-autonomous CRUD POS system.
Supabase (BaaS)	PostgreSQL database, Authentication, and Realtime Websocket broadcasting.	**Medium Risk:** Connection pooling limits. Mitigation: Optimize queries and limit the payload size of real-time broadcasts.

5. Project Management & Team Contributions

5.1. Project Timeline (10-Day Sprint)







5.2. Team Members Role

Tasks	Members	Description & Responsibilities
AI Agent Layer	**Lee Hom, Jian Hui**	LangGraph orchestration, prompt engineering, Dual-Agent debate logic, and unstructured data ingestion mapping.
Backend & DB	**David, Ahmad**	FastAPI microservices, Supabase schema design (3NF), CRUD operations, and enforcing boundary logic constraints.
Frontend & UI	**Ahmad, Nicholas**	React/Next.js development, God-Mode dashboard, real-time SSE syncing, and animated order queue visualizations.

5.3. Recommendations for Future Scalability

- **Redis Caching:** Implement a Redis cache layer in front of the menu_items database. As the primary customer-facing endpoint, serving the menu from cache will dramatically reduce database hits during peak ordering hours.

- **Agent Task Queuing:** Move LangGraph execution from synchronous API calls to a message broker (e.g., Celery/RabbitMQ) to ensure the FastAPI server never blocks while waiting for the LLM to finish thinking.